

Appunti di Core Java 21

Emanuele Romano

Indice

1	Fondamenta	1
1.1	Editing, Compilazione ed Esecuzione	1
1.2	Cenni alla JShell: L'approccio REPL	1
1.3	Tipi Primitivi	2
1.3.1	Il tipo char e Unicode	2
1.3.2	Il tipo boolean	3
1.3.3	Quadro generale dei tipi primitivi	3
1.4	Stringhe	4
1.4.1	Eccezioni allo String Pool: Stringhe Dinamiche	4
1.4.2	L'Immutabilità in Java: Oltre le Stringhe	5
1.5	La Classe StringBuilder	6
1.6	Input e output su console	7
1.6.1	Scanner	7
1.6.2	Output su Console	7
1.7	Lettura e Scrittura su File	8
1.7.1	Lettura da File	8
1.7.2	Scrittura su File	8
1.7.3	Gestione dei Percorsi e Eccezioni	9
1.8	Il Costrutto Switch	9
1.8.1	Lo Switch Tradizionale (Statement)	9
1.8.2	La Sintassi a Freccia (Modern Switch Statement)	9
1.8.3	Switch Expressions	10
1.8.4	Evoluzioni Avanzate: Pattern Matching e Null	10
1.8.5	L'Utilizzo della Clausola <code>when</code> negli <code>switch</code>	11
1.9	Big Numbers: BigInteger e BigDecimal	11
2	Classi e oggetti	13
2.1	I Costruttori	13
2.1.1	Ulteriori note su <code>this</code>	14
2.2	Overloading e Firma dei Metodi	14
2.3	Costruzione degli Oggetti	15
2.3.1	Inizializzazione e Valori di Default	16
2.3.2	Blocchi di Inizializzazione	16
2.3.3	L'Ordine di Esecuzione	16
2.4	L'inferenza del tipo: la parola chiave <code>var</code>	16
2.5	Ulteriori note sull'incapsulamento	17
2.5.1	Il rischio della fuga di riferimenti (<i>Leaking References</i>)	17
2.6	I Metodi Privati	18
2.7	Campi di istanza <code>final</code>	19
2.8	Membri Statici: Campi e Costanti	19
2.8.1	Campi Statici (Variabili di Classe)	19
2.8.2	I Metodi Statici	20
2.9	Il Metodo <code>main</code>	20

2.9.1	Novità Java 21: Instance Main Methods (Preview)	21
2.10	Il passaggio dei parametri: Call-by-Value	21
2.11	Metodi con numero variabile di argomenti (Varargs)	22
2.12	Tipi Enumerati (Enum)	23
2.13	I Records: classi come contenitori di dati	25
2.14	Gestione della Memoria: Stack vs Heap	27
2.14.1	Confronto Sintetico	27
3	Aspetti di Gestione del Codice	29
3.1	Anatomia di un Progetto Java: Convenzioni e Struttura	29
3.2	I Package e il Naming Convention	29
3.2.1	Collaborazione e Conflitti	30
3.3	Creazione e Gestione Manuale dei Package	31
3.4	L'Accesso a livello di Package	32
3.4.1	Sicurezza e Moduli	33
3.5	Gli Import	33
3.5.1	Risoluzione dei Conflitti di Omonimia	34
3.5.2	Gli Import Statici	34
4	Ereditarietà e Polimorfismo	36
4.1	Classi, Superclassi e Sottoclassi	36
4.2	Sovrascrivere i Metodi (Overriding)	37
4.2.1	L'importanza di <code>@Override</code>	38
4.3	I Costruttori delle Sottoclassi	39
4.4	Polimorfismo	40
4.4.1	Il Legame Dinamico (Dynamic Binding)	41
4.4.2	Sintesi Mnemonica: Il Senso di Marcia	42
4.4.3	Covarianza dei Tipi di Ritorno	42
4.5	Prevenire l'Ereditarietà: Il modificatore <code>final</code>	43
4.6	Il Casting degli Oggetti	45
4.7	Evoluzione di <code>instanceof</code> : Pattern Matching	45
4.7.1	Scoping e Operatori Logici	46
4.7.2	Il Record Pattern e la Decostruzione (Java 21+)	47
4.8	Accesso Protetto: Il modificatore <code>protected</code>	48
4.9	La Classe <code>Object</code>	49
4.9.1	Il Metodo <code>equals()</code>	49
4.9.2	Il Metodo <code>hashCode()</code>	53
4.9.3	Il Metodo <code>toString()</code>	55
4.9.4	Il caso particolare dei Records	55
4.10	Classi Astratte	56
4.11	Classi Sigillate (<i>Sealed Classes</i>)	57
5	Interfacce, Lambdas e Inner Classes	58
5.1	Interfacce in Java	58
5.1.1	L'interfaccia <code>Comparable</code>	59
5.1.2	Interfacce vs Classi Astratte	62
5.1.3	Metodi Default: Evoluzione e Compatibilità	63
5.2	Il Pattern Callback: Passare Comportamento	63
5.2.1	Un esempio di Callback: l'interfaccia <code>Comparator</code>	64

5.3	Lambda Expressions	65
5.3.1	Introduzione e Motivazioni	65
5.3.2	Sintassi delle Lambda Expressions	66
5.3.3	Uso corretto delle Lambda Expressions	67
5.3.4	Interfacce Funzionali Standard	69
5.3.5	Method e Constructor References	70
5.3.6	Accesso alle variabili (Variable Capture)	71
5.4	Inner Classes	72
5.4.1	Struttura e Accesso allo Stato	72
5.4.2	Local Inner Classes	74
5.4.3	Meccanica della Variable Capture	75
5.4.4	Classi Anonime	76
5.4.5	Classi Statiche	76
5.4.6	Il Pericolo del Memory Leak nelle Inner Classes	77
5.5	Approfondimento: le Chiusure (Closures)	78
6	Eccezioni, Asserzioni e Logging	80
6.1	Gestione delle Eccezioni	80
6.1.1	La Gerarchia delle Eccezioni	82
6.1.2	Dichiarare e Lanciare Eccezioni	83
6.1.3	Il Blocco <code>try-catch</code> : gestione locale	83
6.1.4	La clausola <code>throws</code> : gestione delegata	85
6.1.5	Ereditarietà ed Eccezioni	86
6.1.6	Creare Classi di Eccezione Personalizzate	87
6.1.7	Exceptions Chaining	87
6.1.8	<code>try-with-resources</code>	88
6.1.9	Costruttori e Metodi di Eccezioni	91
6.2	Le Asserzioni (Assertions)	91
6.2.1	Attivare/Disattivare Asserzioni	92
6.2.2	Quando scegliere le Asserzioni	94
6.3	Logging [BOZZE DA RIPRENDERE]	95
6.3.1	Architettura del Logging: Facciate e Implementazioni	95
6.3.2	L'interfaccia <code>System.Logger</code>	96
6.3.3	Ottenimento e Utilizzo del Logger	97
7	Generics	99
7.1	Introduzione	99
7.2	Struttura di una Classe Generica	100
7.3	Metodi Generici	101
7.4	Variabili di Tipo Vincolate (Bounded Type Variables)	101
7.5	Funzionamento interno dei Generics	102
7.5.1	Type Erasure (Cancellazione del Tipo)	102
7.5.2	Traduzione delle Espressioni Generiche	103
7.5.3	Type Erasure e Polimorfismo: i Metodi Ponte (Bridge Methods)	103
7.6	Interoperabilità con Codice Legacy	105
7.7	Record Patterns e Generics	106
7.8	Ereditarietà nei Tipi Generici	107
7.9	Wildcard Types (Tipi Jolly)	108
7.9.1	Usi consentiti	108

7.9.2	Wildcard con vincolo inferiore: ? super T	110
7.9.3	Applicazioni in Supertipi Generici	110
7.9.4	Il Principio PECS (Producer Extends, Consumer Super)	112
7.9.5	Unbounded Wildcards	113
7.9.6	Wildcard Capture	113
7.10	Restrizioni e Limitazioni dei Generics	114
7.10.1	Verifiche di Tipo a Runtime	114
7.10.2	Divieto di Creazione di Array Generici	114
7.10.3	Limiti sui Varargs	115
7.10.4	Divieto di Istanziamento di Variabili di Tipo	116
8	Collections	117
8.1	Separazione tra Interfaccia e Implementazione	117
8.2	L'Interfaccia Collection	118
8.2.1	Il Concetto di Operazioni Opzionali	120
8.3	Iteratori	120
8.3.1	L'Interfaccia ListIterator	123
8.3.2	Sicurezza e Iterazione: Perché l'Indice è Pericoloso	124
8.4	Le Classi Astratte "Ponte"	125
8.5	Interfacce della Gerarchia Collection	126
8.5.1	L'Interfaccia List	126
8.5.2	L'Interfaccia Set (Insiemi)	127
8.5.3	SequencedCollection (Java 21+)	128
8.5.4	Queue e Deque	129
8.6	Classi Collection Concrete	130
8.6.1	Il Motore ad Array (Contiguità Fisica)	130
8.6.2	Il Motore a Nodi: LinkedList	130
8.6.3	Il Motore Hash (Tabelle Hash)	131
8.6.4	EnumSet : Prestazioni a Livello di Bit	132
8.6.5	Il Motore ad Albero: TreeSet	132
8.6.6	Il Motore a Heap: PriorityQueue	132
8.7	L'Interfaccia Map	133
A	Appendice A: Nozioni Accessorie	135
A.1	Il Concetto di Hashing	135
A.2	Tabelle Hash in Java	136
A.3	L'Array Circolare (Circular Array)	137
A.4	La Struttura Dati Heap	138
A.5	La Coda a Priorità (Priority Queue)	139

Collections

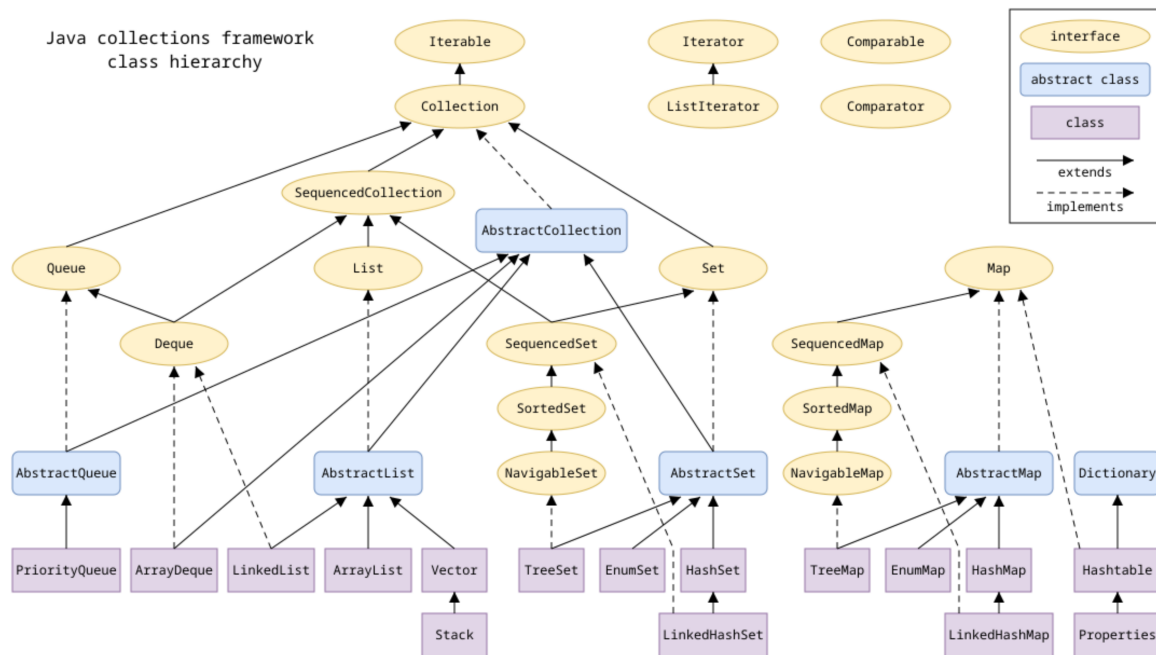


Figura 8.1: Gerarchia delle interfacce del Java Collection Framework

Il *Java Collections Framework* (JCF) si presenta come un'architettura gerarchica vasta e articolata. Per navigare in questo "groviglio" di interfacce e classi, adotteremo un **approccio top-down**: inizieremo l'analisi dalle astrazioni più elevate per poi scendere progressivamente verso le specializzazioni e le implementazioni concrete.

Il punto di partenza è rappresentato dalle interfacce radice, **Iterable** e **Collection**, che stabiliscono i requisiti minimi e i comportamenti comuni a quasi tutti i contenitori di dati in Java.

8.1 Separazione tra Interfaccia e Implementazione

Il *Java Collections Framework* adotta un approccio fondamentale della programmazione a oggetti: la netta separazione tra le **interfacce** (che definiscono il comportamento) e le **implementazioni** concrete (che definiscono le strutture dati e le prestazioni). Per illustrare bene questo concetto e i suoi vantaggi, esaminiamo il seguente esempio.

Una **Queue** (coda) specifica un comportamento **FIFO** (*First In, First Out*): aggiunta in coda, rimozione in testa e monitoraggio della dimensione.

Listing 8.1: Interfaccia minimale di una Queue

```
1 public interface Queue<E> {
```

```

2    void add(E e);
3    E remove();
4    int size();
5 }

```

L'interfaccia non dice nulla su come i dati siano memorizzati. Esistono due implementazioni principali nel JDK:

1. **ArrayDeque**: Utilizza un *circular array* (array circolare, vedi in appendice A.3).
2. **LinkedList**: Utilizza una lista concatenata di nodi.

Il vantaggio principale di questa separazione è la flessibilità. Nel codice backend, è considerata una *best practice* utilizzare il tipo interfaccia per dichiarare le variabili, limitando l'uso della classe concreta solo al momento della costruzione.

Listing 8.2: Utilizzo corretto delle interfacce

```

1 // Uso l'interfaccia come tipo della variabile
2 Queue<Customer> expressLane = new ArrayDeque<>(100);
3 expressLane.add(new Customer("Harry"));
4
5 // Se cambio idea, devo cambiare solo la riga del costruttore
6 Queue<Customer> expressLane = new LinkedList<>();

```

✓ Coding to an Interface

Usare il tipo dell'interfaccia (es. `Queue`) invece del tipo concreto (es. `ArrayDeque`) protegge il codice da dipendenze eccessive. Se accidentalmente si utilizzasse un metodo specifico di `ArrayDeque` non presente in `Queue`, il compilatore segnalerebbe l'errore, garantendo che il codice rimanga facilmente intercambiabile.

In Java 6, quando fu introdotta `ArrayDeque`, i programmatori che avevano "programmato per interfacce" poterono migliorare istantaneamente le prestazioni delle loro applicazioni cambiando una singola riga di codice (il costruttore), lasciando intatto tutto il resto della logica aziendale.

Perché scegliere l'una o l'altra? L'interfaccia è agnostica riguardo alle prestazioni, ma le implementazioni no:

- **ArrayDeque**: È generalmente più performante. Ha un impatto minore sul *Garbage Collector* perché non deve creare un oggetto "Nodo" per ogni elemento inserito. Richiede la riallocazione dell'array quando la capacità finisce (operazione computazionalmente costosa), ma se la capacità iniziale è ben calibrata, queste riallocazioni sono rare.
- **LinkedList**: Non ha problemi di capacità finita, ma ogni elemento aggiunto comporta la creazione di un nuovo oggetto in memoria, rendendola meno efficiente in scenari ad alto volume di dati.

8.2 L'Interfaccia Collection

L'interfaccia `Collection<E>` è la radice della gerarchia per le strutture dati che gestiscono gruppi di oggetti (ad eccezione delle *Map*). Essa definisce un set di operazioni standard che garantiscono l'interoperabilità tra diverse implementazioni. Di seguito, esaminiamo i metodi di uso più comune.

Operazioni di Query (Ispezione) Questi metodi permettono di ottenere informazioni sullo stato della collezione senza modificarla.

- `int size()`: Restituisce il numero di elementi.
- `boolean isEmpty()`: Equivalente a `size() == 0`.
- `boolean contains(Object o)`: Verifica la presenza di un elemento.

Operazioni di Modifica :

- `boolean add(E element)`: Restituisce **true** se l'aggiunta ha effettivamente modificato la collezione, **false** altrimenti. Ad esempio, in una *List*, il metodo `add` restituirà quasi sempre **true**; invece, in un *Set* (che non ammette duplicati), se provi ad aggiungere un oggetto già presente, il metodo restituirà **false**, segnalando che l'operazione non ha avuto effetto.
- `boolean remove(Object o)`: Rimuove una singola istanza dell'elemento, se presente. Il valore restituito segue la stessa logica di `add`.
- `void clear()`: Rimuove tutti gli elementi, svuotando la collezione.

Operazioni Bulk (di Massa) Questi metodi sono fondamentali per manipolare interi set di dati (es. confrontare liste di ID o filtrare risultati). Per `addAll`, `removeAll` e `retainAll` il valore di ritorno indica se la struttura dati è stata mutata, per `containsAll` il boolean rappresenta l'esito logico del test di inclusione.

- `boolean addAll(Collection<? extends E> c)`: **Unione**. Aggiunge tutti gli elementi della collezione *c* alla collezione corrente.
- `boolean removeAll(Collection<?> c)`: **Differenza**. Rimuove dalla collezione corrente tutti gli elementi che sono presenti anche in *c*.
- `boolean retainAll(Collection<?> c)`: **Intersezione**. Rimuove dalla collezione corrente tutti gli elementi che **non** sono contenuti in *c*, mantenendo solo i comuni.
- `boolean containsAll(Collection<?> c)`: **Inclusione**. Verifica se la collezione corrente contiene tutti gli elementi della collezione *c*.

Conversione in Array Nonostante la potenza delle collezioni, a volte è necessario interfacciarsi con API legacy che richiedono array classici.

- `Object[] toArray()`: Restituisce un array di oggetti.
- `<T> T[] toArray(T[] a)`: Versione tipizzata. Se l'array *a* è abbastanza grande, i dati vengono inseriti lì; altrimenti, viene creato un nuovo array dello stesso tipo.

Metodi Default e Approccio Funzionale (Java 8+) Con l'introduzione delle lambda, l'interfaccia si è arricchita di metodi **default** molto potenti:

- `boolean removeIf(Predicate<? super E> filter)`: Rimuove tutti gli elementi che soddisfano una determinata condizione. È molto più efficiente e pulito di un ciclo manuale con iteratore.
- `Stream<E> stream()`: Il punto d'ingresso per la *Stream API*, essenziale per trasformazioni e filtraggi complessi in stile dichiarativo. Tratteremo gli *Stream* in un capitolo successivo.



Rischio di Errori a Runtime per Parametri Legacy

Un dettaglio tecnico sottile ma cruciale riguarda il tipo dei parametri di alcuni metodi fondamentali come `contains`, `remove` e le operazioni bulk. Sebbene la

⚠ collezione sia generica (`Collection<E>`), questi metodi accettano parametri di tipo `Object` o `Collection<?>`.

Il *Java Collections Framework* è stato progettato prima dell'introduzione dei tipi generici. Per mantenere la compatibilità con il codice legacy, i parametri di `contains` e `remove` sono rimasti di tipo `Object`. Questo significa che il compilatore non segnalerà errori se si tenta di cercare o rimuovere un oggetto di tipo errato.

Listing 8.3: Esempio di errore di tipo non rilevato

```
1  Collection<Path> paths = ...;
2  // Errore logico: cerco di rimuovere una String da una
   collezione di Path
3  paths.remove("/tmp"); // Compila correttamente, ma non
   avra' alcun effetto
```

8.2.1 Il Concetto di Operazioni Opzionali

Quasi tutti i metodi che abbiamo elencato non sono di default e ciò potrebbe indurre a pensare che *tutte* le collezioni concrete in Java implementino tutti questi metodi. Tuttavia, questo può essere fuorviante: il framework Java gestisce la varietà delle strutture dati (es. collezioni a dimensione fissa o non modificabili) attraverso il concetto di **metodi opzionali**.

Invece di creare decine di sotto-interfacce, i progettisti di Java hanno preferito mantenere l'interfaccia `Collection` unica. Se una specifica implementazione non supporta un metodo (ad esempio il metodo `add` in una collezione a sola lettura), il metodo viene comunque implementato, ma il suo corpo consiste nel lancio di una eccezione:

```
1 public boolean add(E e) {
2     throw new UnsupportedOperationException();
3 }
```

⚠ Attenzione a Runtime

Poiché l'obbligo di implementazione è soddisfatto formalmente, il compilatore non segnalerà errori. L'errore emergerà solo a **runtime**. Questo è un motivo in più per cui è fondamentale conoscere bene la documentazione della classe concreta che si sta istanziando.

8.3 Iteratori

Per scorrere gli elementi di una collezione, il Java Collections Framework non si affida agli indici (che sarebbero inefficienti per strutture come le liste concatenate), ma utilizza l'interfaccia `Iterator<E>`; essa definisce il meccanismo standard per visitare gli elementi di una collezione uno alla volta. A livello astratto, si può immaginare l'iteratore come un cursore posizionato **tra** gli elementi (si veda la figura 8.2).

L'interfaccia `Iterator` definisce quattro metodi principali:

```
1 public interface Iterator<E> {
2     boolean hasNext();
3     E next();
```

```

4  default void remove();
5  default void forEachRemaining(Consumer<? super E> action);
6  }

```

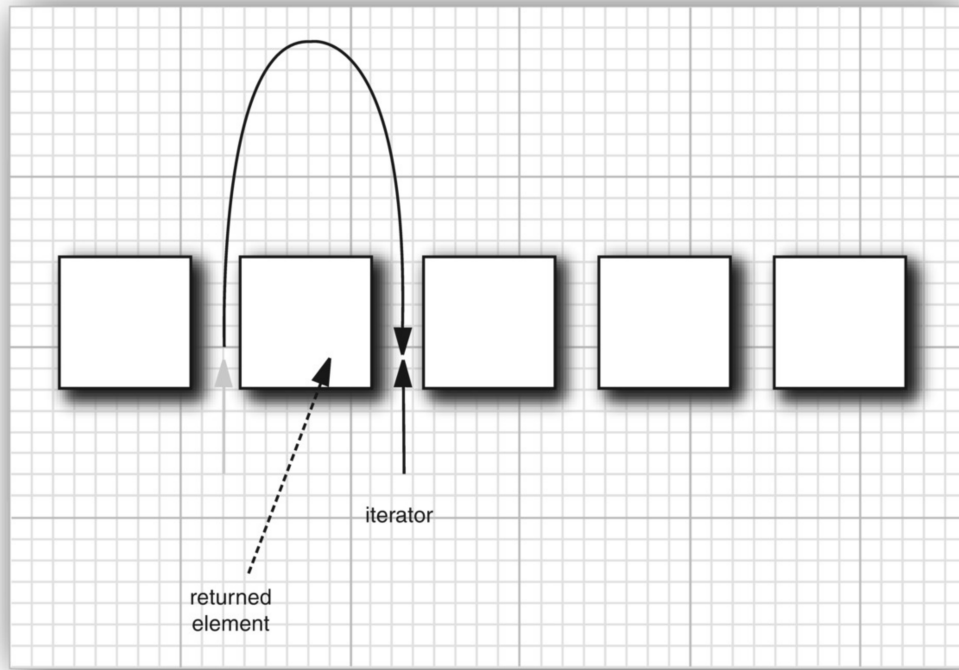


Figura 8.2: l'iteratore come cursore **tra** elementi

- `boolean hasNext()`: Restituisce `true` se ci sono ancora elementi da scorrere.
- E `next()`: Chiamando questo metodo, il cursore "salta" oltre l'elemento successivo e restituisce un riferimento all'elemento che ha appena saltato.
- `void remove()`: Cancella l'elemento restituito dall'ultima chiamata di `next`; esso è soggetto a vincoli: può essere chiamato solo dopo una chiamata a `next` (in concetto è: prima lo devo leggere, poi lo posso cancellare) e solo **una volta** dopo ogni chiamata a `next()` (non si può chiamare due volte di fila senza un `next` in mezzo). Se uno di questi vincoli non viene rispettato, si riceve una `IllegalStateException`.
- `void forEachRemaining(Consumer<? super E> action)`: Esegue un'azione su tutti gli elementi rimanenti.



Da notare che nel codice dell'interfaccia i metodi `remove()` e `forEachRemaining()` sono segnati come default. Questo significa che chi implementa l'interfaccia non è obbligato a scriverne il corpo se non ne ha bisogno.

Nello specifico, `remove()` è un metodo "opzionale": alcune collezioni (come quelle non modificabili) potrebbero lanciare una `UnsupportedOperationException` se provi a usarlo.

Come si vede dalla figura 8.1, `Iterable` rappresenta l'interfaccia padre di `Collection`; ogni classe che la implementa dichiara di poter restituire i propri elementi uno alla volta. Essa contiene un solo metodo astratto e non di default che restituisce un iteratore:

```
1 public interface Iterable<E> {  
2     Iterator<E> iterator();  
3 }
```

Questo metodo viene dunque ereditato da `Collection`; per ispezionare tutti gli elementi di una collezione, si richiede un iteratore e si richiama ripetutamente il metodo `next()`. Se si chiama `next()` quando si è già raggiunta la fine della collezione, l'iteratore solleva una `NoSuchElementException`. Per questo motivo, è fondamentale verificare sempre la presenza di un elemento successivo tramite il metodo `hasNext()`.

Listing 8.4: Utilizzo classico del ciclo while con Iterator

```
1 Collection<String> coll = ...;  
2 Iterator<String> iter = coll.iterator();  
3 while (iter.hasNext()) {  
4     String element = iter.next();  
5     // operazione sull'elemento  
6 }
```

Java permette di scrivere il ciclo precedente in modo molto più conciso grazie al costrutto **for each**. Il compilatore traduce automaticamente questo costrutto in un ciclo con iteratore.

Listing 8.5: Sintassi for each

```
1 for (String element : coll) {  
2     // operazione sull'elemento  
3 }
```

Q Dettagli

```
1 // Scritto dallo sviluppatore:  
2 for (String s : coll) { System.out.println(s); }  
3  
4 // Tradotto dal compilatore:  
5 Iterator<String> it = coll.iterator();  
6 while (it.hasNext()) {  
7     String s = it.next();  
8     System.out.println(s);  
9 }
```

Il ciclo "for each" può essere utilizzato con qualsiasi oggetto che implementi l'interfaccia `Iterable`.



Poiché l'interfaccia `Collection` estende `Iterable`, ogni collezione del framework Java può essere scorsa usando la sintassi *for each*. Prediligi sempre questa sintassi per la leggibilità, a meno che tu non abbia bisogno di rimuovere elementi durante

✓ l'iterazione.

⚠ Effetti Collaterali

Tieni presente che durante un `forEach`, non puoi modificare la struttura della collezione (es. aggiungere o rimuovere elementi), altrimenti Java lancerà una `ConcurrentModificationException`.

Oltre ai cicli tradizionali, il *Collections Framework* mette a disposizione metodi che accettano **lambda expressions** per elaborare gli elementi in modo più conciso e leggibile. Invece di gestire manualmente l'iteratore, possiamo passare un'azione (sotto forma di lambda) direttamente alla collezione o all'iteratore stesso.

Listing 8.6: Esempi di iterazione funzionale

```
1 // Applica l'azione a ogni elemento della collezione
2 myCollection.forEach(element -> System.out.println(element));
3
4 // Applica l'azione agli elementi rimanenti dell'iteratore
5 myIterator.forEachRemaining(element -> System.out.println(element
  ));
```

Entrambi questi metodi utilizzano l'interfaccia funzionale `Consumer<? super E>`. La lambda viene invocata internamente per ogni elemento della collezione senza che lo sviluppatore debba preoccuparsi di chiamare `next()` o `hasNext()`.

Per quanto riguarda l'ordine in cui gli elementi vengono visitati, il comportamento dipende esclusivamente dal tipo di collezione; ad esempio, per un `ArrayList` l'iteratore è **predicibile**: inizia dall'indice 0 e incrementa di uno a ogni passo; per un `HashSet`, l'iteratore visita gli elementi in ordine **essenzialmente casuale** e non si può fare alcuna assunzione sulla sequenza di uscita.

Se in una collezione l'iteratore è predicibile, si dice che ha un **encounter order** (ordine di incontro).

💡 Scelta della Collezione

Se l'ordine dei dati è irrilevante (es. sommare dei totali o contare occorrenze), l'incertezza sull'ordine di visita non è un problema. Se invece l'ordine è parte della logica di business (es. una lista di transazioni cronologiche), è vitale scegliere una collezione che garantisca l'ordinamento come `ArrayList` o `TreeSet`.

8.3.1 L'Interfaccia `ListIterator`

Mentre `Iterator` permette solo lo scorrimento in avanti, **`ListIterator`** è una specializzazione per le liste che permette la navigazione bidirezionale e la modifica della struttura durante l'iterazione. **Implementazioni:** È fornita da tutte le classi della gerarchia `List` (`ArrayList`, `LinkedList`, `Vector`)

⚠ L'importanza in `LinkedList`

Metodo	Descrizione
<code>next()</code> / <code>previous()</code>	Sposta il cursore e restituisce l'elemento saltato.
<code>hasNext()</code> / <code>hasPrevious()</code>	Verifica la presenza di elementi nelle due direzioni.
<code>add(e)</code>	Inserisce un elemento nella posizione del cursore.
<code>set(e)</code>	Sostituisce l'ultimo elemento restituito da <code>next/previous</code> .
<code>nextIndex()</code> / <code>previousIndex()</code>	Restituisce l'indice dell'elemento che verrebbe restituito.

Tabella 8.1: Metodi principali di `ListIterator`

In una `LinkedList`, l'unico modo efficiente ($O(1)$) per inserire o rimuovere ripetutamente elementi nel mezzo della lista è posizionarsi con un `ListIterator` nella posizione desiderata. Senza di esso, ogni operazione di accesso richiederebbe uno scorrimento lineare $O(n)$ dalla testa della lista ogni singola volta.

**Tip**

Capita raramente di usare `previous()`, ma non è così raro usare `ListIterator` per trasformare una lista mentre la si scorre. L'iteratore standard permette solo di rimuovere (`remove()`), mentre il `ListIterator` permette di sostituire un elemento (`set()`) o aggiungerne uno nuovo (`add()`) esattamente in quel punto, senza mandare in crash la collezione con una `ConcurrentModificationException`.

**Tip**

Sebbene per `ArrayList` sia possibile sostituire elementi tramite indice senza causare eccezioni, l'uso di `ListIterator` rimane fondamentale per ragioni di Sicurezza Strutturale: Permette di aggiungere (`add()`) o rimuovere elementi durante l'iterazione. Mentre `myList.add()` causerebbe una `ConcurrentModificationException` in un ciclo *for-each*, `myIterator.add()` aggiorna correttamente lo stato dell'iteratore.

8.3.2 Sicurezza e Iterazione: Perché l'Indice è Pericoloso

La scelta del ciclo per scorrere una collezione non è solo una questione di stile o di "zucchero sintattico", ma una decisione ingegneristica che impatta sulla robustezza del codice.

L'utilizzo di un indice (`for(int i=0; i < list.size(); i++)`) è considerato **pericoloso** se la lista viene modificata durante il ciclo.

- **Problema:** Quando si rimuove l'elemento all'indice i , tutti gli elementi successivi scalano a sinistra. Al prossimo incremento ($i++$), il ciclo salterà l'elemento che è appena scalato nella posizione i .
- **Comportamento:** Il codice non crasha, ma produce risultati errati e imprevedibili (*silent failure*).

Nonostante la superiorità dell'iteratore per la sicurezza, il ciclo `for` con indice rimane lo strumento d'elezione in scenari specifici.

1. **Sincronizzazione di più Collezioni:** Quando è necessario accedere a due o più strutture dati simultaneamente utilizzando la stessa posizione relativa.
2. **Calcoli basati sulla Posizione:** Quando l'indice i è un parametro della formula matematica o della logica di business.

3. **Accesso Non-Sequenziale:** Algoritmi che richiedono salti ($i += k$), scansioni a ritroso o partizionamento (es. *Binary Search*, *QuickSort*).
4. **Prestazioni su Primitivi:** Negli array di tipi primitivi, evita l'overhead di creazione degli oggetti *Iterator* e favorisce le ottimizzazioni della CPU.

 Tip

Per scorrere qualsiasi tipo di struttura dati, prediligi l'uso di iteratori oppure, per la sola lettura, *enhanced for*.

Usa il ciclo con indice come un **bisturi**: quando la struttura dati è un array semplice o quando la logica dell'algoritmo non è una semplice "visita" di tutti gli elementi, ma una manipolazione basata sulla loro coordinata spaziale.

8.4 Le Classi Astratte "Ponte"

Per evitare che gli sviluppatori debbano scrivere decine di metodi ogni volta che creano una nuova collezione, il JDK fornisce delle classi astratte chiamate ***Abstract Skeleton Implementations***:

- `AbstractCollection`
- `AbstractList`
- `AbstractSet`

Queste classi implementano già la maggior parte dei metodi (come `containsAll`, `removeAll`, `toString`) basandosi sui metodi fondamentali `iterator` e `size`.

 Tip

Se un giorno dovessi creare una tua collezione personalizzata, non implementare direttamente l'interfaccia `Collection`. Estendi invece `AbstractCollection`: dovrai implementare solo `iterator()` e `size()`, e avrai tutti gli altri metodi (compresi quelli bulk) già pronti e funzionanti.

Per capire come queste classi riescano a implementare metodi complessi senza sapere come sono memorizzati i dati, osserviamo come `AbstractCollection` implementa il metodo `contains(Object o)`.

Listing 8.7: Logica di `AbstractCollection.contains`

```
1 public boolean contains(Object o) {
2     Iterator<E> it = iterator(); // Metodo astratto!
3     if (o == null) {
4         while (it.hasNext())
5             if (it.next() == null) return true;
6     } else {
7         while (it.hasNext())
8             if (o.equals(it.next())) return true;
9     }
10    return false;
11 }
```

Osservazione 8.1. `AbstractCollection` non ha idea se la collezione sia un array o una lista concatenata. Tuttavia, sa che ogni collezione **deve** fornire un `Iterator`. Usando quel-

l'unico metodo (che rimane astratto nella classe `skeleton`), può fornire un'implementazione funzionante di `contains`, `toArray`, `toString` e molti altri.

Quando una classe concreta estende `AbstractCollection`, il carico di lavoro dello sviluppatore è ridotto, ma non nullo.

- **Metodi Obbligatori:** La classe concreta deve fornire le implementazioni di `iterator()` e `size()`.
- **Collezioni Mutabili:** Se la collezione deve permettere la modifica, è necessario implementare anche il metodo `add()`. In caso contrario, il metodo `add()` ereditato lancerà una `UnsupportedOperationException`.
- **Ottimizzazione:** Sebbene `AbstractCollection` fornisca già le implementazioni di molti metodi concreti, la classe figlia è libera di sovrascriverli se può offrire un'implementazione più efficiente (ad esempio, sfruttando un indice o una tabella hash interna per evitare lo scorrimento lineare).

Q Dettagli

L'approccio basato sulle "classi ponte" è oggi considerato parzialmente superato dal punto di vista del design. Sarebbe stato preferibile avere questi metodi direttamente nell'interfaccia `Collection` come metodi `default`, ma questa funzionalità non esisteva quando il framework fu progettato.

Tuttavia, nelle versioni recenti sono stati aggiunti diversi metodi `default`:

- **Stream API:** Metodi per l'elaborazione dei dati (trattati nel Volume II).
- **Conversione:** Metodi `toArray` più flessibili.
- **Rimozione Condizionale:** Il metodo `removeIf`.

8.5 Interfacce della Gerarchia `Collection`

Esaminiamo ora alcune specifiche interfacce della parte del framework che si dirama da `Collection`.

8.5.1 L'Interfaccia `List`

Una `List` è una collezione ordinata in cui ogni elemento ha una posizione precisa. L'accesso agli elementi può avvenire in due modi:

1. **Accesso Sequenziale:** Utilizzando un iteratore per visitare gli elementi uno dopo l'altro.
2. **Accesso Casuale (Random Access):** Utilizzando un indice intero per saltare direttamente a una posizione specifica (come in un array).

Per l'accesso casuale, l'interfaccia definisce metodi specifici per manipolare la lista tramite indici:

- `void add(int index, E element):` Inserisce un elemento in una posizione specifica.
- `E remove(int index):` Rimuove e restituisce l'elemento alla posizione indicata.
- `E get(int index):` Restituisce l'elemento alla posizione indicata.
- `E set(int index, E element):` Sostituisce l'elemento in una posizione e restituisce quello precedente.

⚠ L'ambiguità di `remove` con `List<Integer>`

Esiste un'insidia comune quando si usa una lista di interi (`List<Integer>`). Esistono due versioni del metodo `remove`:

- `remove(int index)`: Rimuove l'elemento all'indice specificato.
- `boolean remove(Object o)`: Rimuove la prima occorrenza dell'oggetto specificato.

Se passi un valore `int`, Java sceglierà **sempre** la versione che accetta l'indice, senza eseguire l'autoboxing a `Integer`. Per rimuovere l'elemento con valore 0 (e non l'elemento all'indice 0), devi scrivere: `list.remove(Integer.valueOf(0))`.

🔍 Dettagli

Horstmann evidenzia un difetto strutturale: l'interfaccia `List` raggruppa collezioni basate su array (veloci nell'accesso casuale) e collezioni basate su liste concatenate (lente nell'accesso casuale). Probabilmente sarebbe stato meglio usare due interfacce diverse per i due tipi.

Per permettere agli algoritmi di distinguere queste due tipologie a runtime, Java 1.4 ha introdotto l'interfaccia `RandomAccess`. Si tratta di un'*interfaccia marcatore* (*tagging interface*) che non contiene metodi e serve solo a "etichettare" classi come `ArrayList` per segnalare che supportano l'accesso a tempo costante. È possibile testarla con l'operatore `instanceof`:

```

1      if (list instanceof RandomAccess) {
2          // Posso usare un ciclo for classico con get(i)
3          //      -> O(1)
4      } else {
5          // Meglio usare un iteratore o for-each -> O(n)
6          //      complessivo
7      }

```

8.5.2 L'Interfaccia Set (Insiemi)

L'interfaccia `Set<E>` definisce una collezione che non ammette duplicati. Sebbene dichiari gli stessi metodi di `Collection`, la distinzione è puramente **concettuale** e **semantica** (cambia il *contratto*):

1. Permette al programmatore di scrivere metodi che accettano esplicitamente *solo* insiemi, forzando l'unicità dei dati a livello di tipo.
2. Definisce un criterio di uguaglianza diverso da quello della `List`: per un set l'ordine è un dettaglio irrilevante per l'identità della collezione.

Per oggetti di tipo `Set` devono quindi valere:

- **Rifiuto dei duplicati**: Il metodo `add(E element)` deve garantire che l'elemento venga aggiunto solo se non è già presente. Se l'elemento esiste già, il metodo restituisce `false` e non modifica la collezione.
- **Uguaglianza (`equals`)**: Due set sono considerati uguali se contengono gli stessi elementi, indipendentemente dall'ordine in cui sono stati inseriti.
- **Coerenza Hash**: Il metodo `hashCode()` deve essere implementato in modo che due set con gli stessi elementi generino lo stesso codice hash.

 Il test di uguaglianza

Il metodo `equals` di una classe che implementa `Set` deve verificare che l'oggetto passato come argomento sia a sua volta un `Set`. Non è sufficiente che sia una `Collection` generica; questo protegge dalla possibilità che un `Set` e una `List` (che seguono contratti di uguaglianza diversi) vengano erroneamente considerati uguali.

8.5.3 SequencedCollection (Java 21+)

A partire da Java 21, il framework introduce l'interfaccia `SequencedCollection<E>` per fornire un accesso uniforme al primo e all'ultimo elemento di una collezione che possiede un ordine di incontro (*encounter order*), cioè per cui un iteratore sia **predicibile**.

Prima di questa interfaccia, le operazioni sulla testa e sulla coda della collezione erano frammentate in metodi diversi tra `List`, `Deque` e `SortedSet`. `SequencedCollection` raggruppa queste operazioni sotto un unico contratto.

Listing 8.8: Metodi di `SequencedCollection`

```
1 interface SequencedCollection<E> extends Collection<E> {  
2     // Accesso e modifica  
3     void addFirst(E e);  
4     void addLast(E e);  
5     E getFirst();  
6     E getLast();  
7     E removeFirst();  
8     E removeLast();  
9  
10    // Vista invertita  
11    SequencedCollection<E> reversed();  
12 }
```

Osservazione 8.2. Il metodo `reversed()` non crea una copia della collezione, ma fornisce una **vista** in ordine inverso. Qualsiasi modifica alla collezione originale si riflette nella vista e viceversa, con un'efficienza di $O(1)$.

L'interfaccia `SequencedSet<E>` estende sia `Set` che `SequencedCollection`.

- È implementata da classi che mantengono l'ordine di inserimento (es: `LinkedHashSet`) o l'ordine di ordinamento (es: `TreeSet`).
- Il metodo `reversed()` in questa interfaccia restituisce un `SequencedSet` anziché una `Collection` generica, permettendo di mantenere le proprietà di unicità anche nella vista invertita.

 Se devi scrivere un metodo che accetta qualsiasi cosa sia "ordinata" (lista o set ordinato), considera `SequencedCollection` come tipo del parametro. Questo potrà rendere il codice molto più flessibile.

Da `SequencedSet` si diramano inoltre versioni più sofisticate dell'interfaccia `Set` che vengono utilizzate quando non ci si accontenta dell'unicità, ma si ha bisogno che gli elementi siano mantenuti in un ordine specifico.

- **SortedSet<E>**: Questa interfaccia espone il **Comparator** utilizzato per l'ordinamento e definisce metodi per ottenere "viste" di sotto-insiemi della collezione (es. tutti gli elementi minori di un certo valore).
- **NavigableSet<E>**: Estende **SortedSet** aggiungendo metodi per la ricerca di elementi "vicini" (prossimo o precedente) rispetto a un valore dato.

Osservazione 8.3. Queste interfacce sono implementate dalla classe **TreeSet** che, di fatto, non è altro che un albero binario di ricerca bilanciato (nello specifico, *Red-Black*)

8.5.4 Queue e Deque

Le code sono fondamentali per la gestione di flussi di dati in cui l'ordine di elaborazione è critico.

La **Queue<E>** è progettata per il processamento di elementi in ordine FIFO. L'approccio di Java per le code è molto pragmatico: ogni operazione ha due varianti, una lancia un'eccezione se l'operazione fallisce per effetto di una coda vuota o piena (alcune implementazioni hanno capacità limitata), l'altra restituisce un valore speciale (**null** o **false**).

I metodi **element()**/**peek()** restituiscono la testa della coda senza rimuoverla.

Operazione	Eccezione	Valore Speciale (null/false)
Inserimento	add(e)	offer(e)
Rimozione	remove()	poll()
Ispezione	element()	peek()

Tabella 8.2: Metodi dell'interfaccia Queue

Tip

Molte librerie di messaggistica (come RabbitMQ o le code interne di Spring) usano questi contratti. Sapere che **poll()** non ti manderà in crash l'applicazione se la coda è vuota (a differenza di **remove()**) è la base per scrivere codice robusto che non "esplode" sotto carico.

La **Deque<E>** (Double-Ended Queue) permette l'accesso ad entrambi i capi della collezione. Eredita da **Queue** e aggiunge versioni **First** e **Last** per ogni metodo (es. **addFirst**, **pollLast**).

Deque come Stack

In Java, la classe **Stack** è considerata obsoleta. Se hai bisogno di una struttura LIFO (Last-In-First-Out), il Java Collections Framework raccomanda l'uso di una **Deque**:

- **push(e)** equivale a **addFirst(e)**.
- **pop()** equivale a **removeFirst()**.

Osservazione 8.4. Con l'introduzione di Java 21, **Deque** estende, oltre a **Queue**, anche **SequencedCollection**, rendendo i suoi metodi **getFirst**, **getLast**, ecc., coerenti con il resto del framework.

8.6 Classi Collection Concrete

In questo paragrafo abbandoniamo l'astrazione delle interfacce per analizzare come Java memorizza fisicamente i dati. La scelta della classe concreta influisce drasticamente sulla complessità algoritmica e sull'efficienza della memoria.

Abbiamo già accennato ad alcune di queste classi, qui riepiloghiamo quanto detto e completiamo con ulteriori classi non ancora menzionate.

8.6.1 Il Motore ad Array (Contiguità Fisica)

Le classi in questo gruppo utilizzano un array interno. La caratteristica principale è la **contiguità della memoria**, che favorisce la velocità di accesso e l'efficienza della cache della CPU. Sia `ArrayList` che `ArrayDeque` si basano su un array che viene ridimensionato quando la capacità massima viene raggiunta. Quando l'array interno è pieno, Java ne alloca uno nuovo più grande e vi sposta gli elementi. Sebbene la singola operazione di copia sia $O(n)$, essa avviene così raramente che, spalmata su migliaia di inserimenti, il costo medio (costo **ammortizzato**) rimane $O(1)$.

ArrayList

È l'implementazione di `List` più utilizzata. Quando la capacità massima è raggiunta, cresce solitamente del 50% della capacità corrente.

- **Accesso Casuale:** $O(1)$ tramite indice.
- **Inserimento/Rimozione:** $O(n)$ nel caso peggiore (perché deve "spostare" gli elementi), ma l'aggiunta in coda ha un costo **ammortizzato** di $O(1)$.

ArrayDeque

Introdotta per superare i limiti di `Stack` e `LinkedList`, è una coda a doppia entrata basata su un array circolare. Quando la capacità massima è raggiunta, cresce solitamente del 100% della capacità corrente.



Best Practice

È la scelta più efficiente per implementare sia **Pile** (Stack) che **Code** (Queue). È sensibilmente più veloce di `LinkedList` perché non deve creare un oggetto "nodo" per ogni elemento inserito.

8.6.2 Il Motore a Nodi: LinkedList

Non è altro che una lista doppiamente concatenata (*doubly linked list*): ogni elemento è memorizzato in un nodo che contiene un riferimento al nodo precedente e a quello successivo.

Nonostante sia un classico accademico, nel Java moderno `LinkedList` è raramente la scelta migliore:

1. **Cache Miss:** I nodi sono sparsi nella RAM, obbligando la CPU a continui e lenti recuperi dalla memoria principale.
2. **Overhead:** Ogni nodo richiede memoria extra per due puntatori (`next` e `prev`).
3. **Accesso:** Anche per le operazioni che sembrano $O(1)$, bisogna spesso scorrere la lista per raggiungere il punto desiderato ($O(n)$).

Se `ArrayList` è quasi sempre superiore, quando dovremmo preferire `LinkedList`? La risposta risiede nel costo delle operazioni strutturali durante l'iterazione.

- **Modifiche durante lo scorrimento:** Se il workflow prevede di scorrere una lista con un `ListIterator` e inserire/rimuovere elementi frequentemente nel mezzo, `LinkedList` opera in $O(1)$, mentre `ArrayList` rimane legata a un costo $O(n)$ per lo spostamento dei dati.
- **Assenza di picchi di latenza:** In `LinkedList` non esiste il concetto di "ridimensionamento". Ogni inserimento ha lo stesso costo costante, evitando il picco $O(n)$ tipico della copia degli array dinamici. Questo può essere rilevante se la struttura dati subisce numerosi ridimensionamenti o se si desidera una latenza più prevedibile.



Attenzione: per inserire un elemento in una posizione specifica i , `LinkedList` deve comunque **trovare** quel punto scorrendo i nodi ($O(n)$). Il vantaggio $O(1)$ esiste **solo** se l'iteratore si trova già sulla posizione corretta.

In conclusione, nella stragrande maggioranza delle applicazioni backend (web server, elaborazione dati, microservizi), i benefici della **Cache Locality** dell'array superano di gran lunga i vantaggi teorici dei puntatori. **Usa `ArrayList` di default.**

8.6.3 Il Motore Hash (Tabelle Hash)

Per richiami sulle tabelle hash in Java, si veda [A.2](#)

- **HashSet:** Implementazione standard di `Set` che non garantisce alcun ordine di iterazione. **Prestazioni:** $O(1)$ per ricerca, inserimento e rimozione (se la funzione hash è buona).
- **LinkedHashSet:** Variante di `HashSet` che mantiene una lista doppiamente concatenata tra gli elementi, combinando la velocità del `HashSet` con la capacità di ricordare l'ordine di inserimento. Dunque l'iterazione è predicibile e segue l'ordine in cui gli elementi sono stati aggiunti. È ideale per cache o sistemi dove l'ordine temporale è importante tanto quanto l'unicità.



Mutabilità degli Elementi e Invarianza dell'Hash

Presta estrema attenzione alla modifica (*mutazione*) dello stato degli oggetti dopo averli inseriti in un `Set` (o come chiavi in una `HashMap`).

Se i campi modificati partecipano al calcolo del metodo `hashCode()`, il valore hash dell'oggetto cambierà a seguito della mutazione. Di conseguenza, l'elemento rimarrà fisicamente intrappolato nel bucket associato al suo **vecchio** hash code, ma qualsiasi successiva operazione di ricerca (`contains`), rimozione (`remove`) o inserimento calcolerà l'indice basandosi sul **nuovo** valore hash.

Questo comportamento corrompe l'integrità della struttura dati: l'oggetto modificato diventerà virtualmente "invisibile" alle successive query della mappa o del set, causando fallimenti logici silenziosi (*silent failures*) ed errori di business incredibilmente difficili da tracciare a runtime. Come regola generale, gli elementi inseriti nelle tabelle hash dovrebbero preferibilmente essere **immutabili**.

8.6.4 EnumSet: Prestazioni a Livello di Bit

`EnumSet` è un'implementazione speciale di `Set` progettata per tipi enumerativi. Non è una classe pubblica con costruttori, ma si ottiene tramite metodi factory statici.

- **Meccanismo Interno:** Utilizza un **bit-vector**. Se l'enum ha ≤ 64 elementi (caso comunissimo), il set è memorizzato in un singolo `long`.
- **Efficienza:** Le operazioni sono tradotte in istruzioni bitwise della CPU. È la collezione più veloce in assoluto per i tipi `enum`, con un consumo di memoria minimo.
- **Null-Safety:** Non permette l'inserimento di elementi `null`.

Listing 8.9: Esempio di `EnumSet`

```

1 // Creazione di un set con elementi specifici
2 Set<Day> weekend = EnumSet.of(Day.SATURDAY, Day.SUNDAY);
3
4 // Creazione di un set con tutti gli elementi dell'enum
5 Set<Day> allDays = EnumSet.allOf(Day.class);

```

Osservazione 8.5. L'ordine di iterazione in un `EnumSet` non dipende dall'ordine di inserimento, ma dall'**ordine di dichiarazione** delle costanti nell'Enum originale.

✓ Best Practice

In passato, per gestire i "flag" (es. permessi READ, WRITE, EXECUTE), si usavano costanti intere e maschere di bit manuali ($2^0, 2^1, 2^2$). In Java moderno, `EnumSet` offre la stessa efficienza dei bit ma con la sicurezza dei tipi (Type Safety) dell'orientamento agli oggetti.

8.6.5 Il Motore ad Albero: `TreeSet`

Come già discusso nei paragrafi precedenti, utilizza un albero Red-Black per mantenere gli elementi costantemente ordinati.

- **Prestazioni:** $O(\log n)$.
- **Utilizzo:** Da preferire quando servono operazioni di "range" (es. estrarre tutti i valori tra x e y) o quando l'ordinamento naturale è un requisito.

8.6.6 Il Motore a Heap: `PriorityQueue`

Rappresenta una *coda a priorità* implementata tramite un array dinamico che funge da *Heap* (per entrambi i concetti, si vedano in appendice i richiami A.4 e A.5).

- **Criterio di Priorità:** Se non viene fornito un `Comparator`, gli elementi devono implementare `Comparable` (ordinamento naturale).
- **Min-Priority di default:** In Java, il valore più piccolo è quello con priorità più alta (testa della coda). Per una Max-Priority Queue, occorre invertire il comparatore: `Collections.reverseOrder()`.

L'efficienza della `PriorityQueue` deriva direttamente dall'uso dello heap come struttura dati sottostante:

✓ Efficienza di Massa

Se devi trasformare una collezione di n elementi in una `PriorityQueue`, non fare n

Operazione	Complessità	Nota Tecnica
<code>peek()</code>	$O(1)$	La radice dello heap è sempre accessibile.
<code>offer()/add()</code>	$O(\log n)$	Richiede il "galleggiamento" (<i>sift-up</i>).
<code>poll()</code>	$O(\log n)$	Richiede l'affondamento (<i>sift-down</i>).
<code>remove(Object)</code>	$O(n)$	Attenzione: richiede la ricerca lineare.
<code>contains(Object)</code>	$O(n)$	Lo heap non è ottimizzato per la ricerca.
<code>heapify</code> (Costruttore)	$O(n)$	Algoritmo di Floyd (costruzione in massa).

Tabella 8.3: Complessità temporale della PriorityQueue

- ✓ inserimenti ($O(n \log n)$). Usa il costruttore che accetta una collezione: Java userà l'algoritmo di **Floyd** (*heapify*) che costruisce lo heap in tempo $O(n)$.

⚠ Gestione dei pareggi

A differenza di quanto si potrebbe pensare, la `PriorityQueue` di Java **non garantisce** un ordine FIFO per elementi con la stessa priorità. Se l'ordine di arrivo è cruciale in caso di parità, è necessario includere un "timestamp" o un contatore nel comparatore. Stesso discorso vale per l'ordine di iterazione.

⚠ Modifica degli elementi

Se un oggetto già presente nella coda viene modificato in modo da cambiare la sua priorità (il risultato di `compareTo` cambia), lo heap **non si riposiziona automaticamente**. Questo corrompe la struttura dati. In questi casi, l'elemento deve essere rimosso e reinserito.

8.7 L'Interfaccia Map

L'interfaccia `Map<K, V>` rappresenta una struttura dati che non fa parte della gerarchia `Collection`. Invece di memorizzare elementi singoli, una mappa gestisce associazioni tra **Chiavi** (Key) e **Valori** (Value). Il contratto fondamentale di una mappa stabilisce che le chiavi devono essere **uniche** (non sono ammessi duplicati), mentre i valori possono essere ripetuti. Di seguito, esaminiamo i metodi di uso più comune, raggruppati in parallelo con quelli delle collezioni.

Operazioni di Query (Ispezione) Questi metodi permettono di ottenere informazioni sullo stato della mappa e di cercare elementi senza modificarla. Notare che la ricerca è divisa tra chiavi e valori.

- `int size()`: Restituisce il numero di associazioni chiave-valore presenti.
- `boolean isEmpty()`: Equivalente a `size() == 0`.
- `boolean containsKey(Object key)`: Verifica se la mappa contiene la chiave specificata. È un'operazione estremamente veloce ($O(1)$ nella maggior parte delle implementazioni).
- `boolean containsValue(Object value)`: Verifica se la mappa contiene una o più istanze del valore specificato. Spesso richiede una ricerca lineare ($O(n)$).

- **V** `get(Object key)`: Restituisce il valore associato alla chiave, oppure `null` se la chiave non è presente.

Operazioni di Modifica A differenza delle collezioni, i metodi di modifica delle mappe restituiscono spesso il valore precedentemente associato alla chiave, fornendo un feedback immediato sulle sostituzioni.

- **V** `put(K key, V value)`: Inserisce una nuova associazione. Se la chiave era già presente, il vecchio valore viene sovrascritto e **restituito** dal metodo. Se la chiave è nuova, restituisce `null`.
- **V** `remove(Object key)`: Rimuove l'associazione per la chiave specificata e restituisce il valore che è stato appena rimosso (o `null` se non esisteva).
- **void** `clear()`: Rimuove tutte le associazioni, svuotando la mappa.

Operazioni Bulk (di Massa) Per le mappe, le operazioni di massa si concentrano principalmente sull'unione di dizionari diversi.

- **void** `putAll(Map<? extends K, ? extends V> m)`: **Unione**. Copia tutte le associazioni dalla mappa `m` alla mappa corrente. Se ci sono chiavi coincidenti, i valori della mappa `m` sovrascrivono quelli attuali.

Le "Viste" (Views) della Mappa Poiché una mappa non è una `Collection` e non possiede un iteratore proprio, per scorrere i suoi elementi è necessario estrarre una delle sue tre "viste". Queste viste sono supportate dalla mappa originale: le modifiche alla vista si riflettono sulla mappa e viceversa.

- **Set<K>** `keySet()`: Restituisce un set contenente tutte le chiavi (è un **Set** perché le chiavi sono uniche).
- **Collection<V>** `values()`: Restituisce una collezione contenente tutti i valori (è una generica `Collection` perché i valori possono contenere duplicati).
- **Set<Map.Entry<K, V>** `entrySet()`: Restituisce un set di oggetti `Map.Entry`. Ogni `Entry` rappresenta la singola coppia inseparabile Chiave-Valore. È il metodo più efficiente per iterare su entrambi contemporaneamente.

Metodi Default e Approccio Funzionale (Java 8+) L'interfaccia `Map` ha beneficiato enormemente dell'introduzione dei metodi `default`, che hanno snellito drasticamente il codice per la gestione dei valori nulli e delle operazioni condizionali.

- **V** `getOrDefault(Object key, V defaultValue)`: Se la chiave esiste, restituisce il suo valore; altrimenti restituisce `defaultValue` senza inserirlo nella mappa. Elimina la necessità di continui controlli `if` (`map.containsKey(...)`).
- **V** `putIfAbsent(K key, V value)`: Inserisce la coppia solo se la chiave non è già mappata a un valore (o se è mappata a `null`).
- **void** `forEach(BiConsumer<? super K, ? super V> action)`: Permette di iterare su chiavi e valori in modo funzionale e pulito tramite una lambda (es. `map.forEach((k, v) -> System.out.println(k + ":" + v))`).
- **V** `computeIfAbsent(K key, Function<? super K, ? extends V> mappingFunction)`: Molto usato per costruire mappe di collezioni o cache. Se la chiave manca, esegue la funzione per calcolare il valore, lo inserisce e lo restituisce in un solo colpo.

Appendice A: Nozioni Accessorie

A.1 Il Concetto di Hashing

L'hashing è uno dei pilastri fondamentali dell'informatica moderna. Non è una tecnologia esclusiva di Java, ma un concetto matematico e logico universale utilizzato per la sintesi e l'identificazione rapida dei dati.

Un **hash** (o *digest*) è il risultato di una funzione che prende un input di lunghezza arbitraria (come un testo, un file o un oggetto complesso) e lo trasforma in una stringa o in un numero di **lunghezza fissa**.

L'analogia più calzante è quella dell'**impronta digitale**:

- Un libro di 1000 pagine è un dato complesso.
- Il suo valore hash è un numero (es. 42) che rappresenta in modo quasi univoco quel libro.
- È molto più facile confrontare due numeri (42 vs 43) che confrontare due libri parola per parola.

Affinché una funzione di hash sia utile, deve rispettare tre criteri:

1. **Deterministicità**: Lo stesso input deve generare *sempre* lo stesso identico output.
2. **Efficienza**: Il calcolo dell'hash deve essere estremamente rapido, indipendentemente dalla dimensione del dato in ingresso.
3. **Effetto Valanga (*Avalanche Effect*)**: Una minima modifica all'input (anche un singolo bit) deve produrre un hash completamente diverso e non correlato al precedente.

Ambiti di Applicazione

L'hashing risolve problemi critici in diversi settori:

- **Sicurezza delle Password**: I sistemi non memorizzano le password in chiaro, ma solo il loro hash. In caso di furto del database, l'attaccante non può risalire alla password originale.
- **Integrità dei Dati**: Attraverso algoritmi come SHA-256 o MD5, è possibile verificare se un file scaricato è integro o se è stato corrotto durante il trasferimento.
- **Strutture Dati (Hash Tables)**: Permette di trovare un elemento in un insieme di milioni di record in tempo costante ($O(1)$), usando l'hash come indice per un "armadietto" (*bucket*) specifico.

Il Problema delle Collisioni

Poiché l'insieme dei possibili input è infinito, mentre l'insieme dei possibili valori hash (numeri interi o stringhe di lunghezza fissa) è finito, esiste la possibilità teorica che due input diversi producano lo stesso hash. Questo evento è chiamato **collisione**. Un buon algoritmo di hash riduce drasticamente la probabilità di collisioni, e i sistemi informatici (come la JVM di Java) sono progettati per gestire correttamente questi rari casi di "omonimia digitale".

A.2 Tabelle Hash in Java

In Java, una tabella hash è implementata come un **array di liste concatenate** (o *array di nodi*). Ogni singolo slot (o cella) di questo array prende il nome di **bucket** (secchio).

Per determinare l'esatta posizione di un oggetto all'interno della tabella, il sistema esegue, a livello concettuale, due operazioni:

1. Calcola l'hash code dell'oggetto (chiamando il metodo `hashCode()`).
2. Riduce questo numero intero *modulo* la capacità totale dell'array (il numero di bucket).

Il resto della divisione rappresenta l'indice del bucket che ospiterà l'elemento. Ad esempio, se un oggetto ha un hash code pari a 76268 e la tabella ha 128 bucket, l'oggetto verrà posizionato nel bucket all'indice 108 (poiché $76268 \pmod{128} = 108$).

Se si è fortunati, il bucket calcolato risulta vuoto e l'elemento viene semplicemente inserito. Tuttavia, è frequente imbattersi in un bucket già occupato da un altro elemento: questo evento prende il nome di **collisione hash** (*hash collision*). In caso di collisione, il nuovo elemento viene aggiunto alla lista concatenata presente in quel bucket. Durante la ricerca, la JVM dovrà scorrere i nodi di questa lista e utilizzare il metodo `equals()` per confrontare il nuovo oggetto con quelli già presenti, verificandone l'effettiva presenza. Se gli hash code sono distribuiti in modo sufficientemente casuale e il numero di bucket è adeguato, ogni lista conterrà pochissimi elementi, mantenendo il tempo di ricerca medio a $O(1)$.

🔍 La Verità sull'Indice: Bitwise AND e Potenze di 2

La spiegazione matematica tramite l'operatore modulo (%) è corretta sul piano logico, ma **non** corrisponde a ciò che accade fisicamente nella JVM. L'operazione di divisione è computazionalmente costosa per la CPU.

Nel codice sorgente reale del JDK, l'indice non viene calcolato con il modulo, bensì tramite un'operazione binaria estremamente veloce (**Bitwise AND**):

```
1 index = (n - 1) & hash; // dove n e' il numero totale di
   bucket
```

Affinché questa operazione bit a bit restituisca esattamente lo stesso risultato di `hash % n`, esiste un vincolo ingegneristico assoluto: **la capacità dell'array deve sempre essere una potenza di 2** (16, 32, 64, 128, ecc.).

Se in fase di costruzione si passa al costruttore una capacità iniziale arbitraria (es. `new HashMap<>(30)`), Java non creerà un array di 30 elementi, ma lo arrotonderà automaticamente e silenziosamente alla successiva potenza di due più vicina (in questo caso, 32). La capacità di default è fissata a 16.

Ottimizzazioni Interne e Fattore di Carico Sebbene l'hashing garantisca, in media, prestazioni eccellenti ($O(1)$), esistono situazioni critiche che la JVM deve saper gestire, come funzioni hash mal progettate o veri e propri attacchi informatici volti a generare collisioni intenzionali per bloccare il sistema.

A partire da Java 8, se il numero di collisioni in un singolo bucket supera una specifica soglia (attualmente fissata a 8 elementi) e la tabella è sufficientemente grande, la lista concatenata si trasforma automaticamente in un albero binario bilanciato (*Red-Black Tree*). Questo processo degrada le prestazioni in modo controllato a $O(\log n)$, scongiurando il caso peggiore di $O(n)$.

Tuttavia, affinché la JVM possa costruire questo albero in modo efficiente, è fondamentale che gli oggetti utilizzati come chiavi implementino l'interfaccia `Comparable`. In caso contrario, in assenza di un ordinamento naturale esplicito, la JVM dovrà ripiegare su complessi meccanismi di *fallback* basati sugli indirizzi di memoria, penalizzando le prestazioni.

Oltre alla gestione delle singole collisioni, è necessario monitorare il riempimento globale della tabella. Se l'array interno si riempie eccessivamente, la probabilità di collisioni aumenta vertiginosamente. Per evitare questo scenario, interviene un parametro chiamato **Fattore di Carico** (*Load Factor*).

Il fattore di carico è un valore decimale che determina la soglia massima di riempimento prima che la tabella venga ridimensionata. In Java, il valore di default è **0.75**. Ciò significa che quando la tabella è piena al 75% della sua capacità, scatta un'operazione computazionalmente costosa detta **Rehashing**:

1. Viene allocato un nuovo array interno con il doppio della capacità rispetto al precedente.
2. Tutti gli elementi della vecchia tabella vengono ricalcolati (ri-hashati) e inseriti nel nuovo array.
3. La vecchia tabella viene abbandonata per essere poi gestita dal *Garbage Collector*.

Per la maggior parte delle applicazioni backend, il fattore di carico di default (0.75) rappresenta il perfetto compromesso tra efficienza in termini di tempo di esecuzione e occupazione di memoria. Tuttavia, i continui rehashing possono rallentare l'applicazione se non gestiti correttamente.

✓ Inizializzazione Preventiva

Se si conosce in anticipo il volume approssimativo dei dati da gestire, è buona norma inizializzare la tabella con una capacità corretta fin dall'inizio, evitando inutili e pesanti operazioni di rehashing.

La capacità iniziale ottimale non corrisponde al numero esatto degli elementi attesi, ma deve tener conto del fattore di carico:

$$\text{Capacità Ottimale} = \frac{\text{Elementi Attesi}}{\text{Load Factor}}$$

A partire da **Java 19**, il framework mette a disposizione dei comodi *factory methods* statici che eseguono questo calcolo matematico in automatico al posto dello sviluppatore:

```
1 // Il metodo calcola la capacita' ottimale per 10.000
   elementi
2 HashSet<String> mySet = HashSet.newHashSet(10000);
3 HashMap<String, Integer> myMap = HashMap.newHashMap(5000);
```

A.3 L'Array Circolare (Circular Array)

Nello studio delle collezioni, in particolare quando si analizzano le implementazioni di code (*Queue*) e code a doppia entrata (*Deque*), ci si imbatte nel concetto di **array circolare**.

In un normale array lineare, quando implementiamo una coda, aggiungiamo elementi alla fine e li rimuoviamo dall'inizio.

- Ogni volta che rimuoviamo il primo elemento, dovremmo teoricamente "shiftare" tutti gli altri elementi a sinistra per coprire il buco.
- Questa operazione ha una complessità temporale di $O(n)$, rendendo la struttura inefficiente per grandi moli di dati.

L'array circolare risolve il problema "collegando" virtualmente l'ultima posizione dell'array alla prima. Invece di spostare i dati, spostiamo i **puntatori** (indici) che indicano dove inizia e dove finisce la nostra collezione.

L'implementazione si basa su due indici principali:

1. **head**: Punta alla posizione del primo elemento disponibile (per la rimozione).
2. **tail**: Punta alla posizione successiva all'ultimo elemento (per l'inserimento).

Quando un indice raggiunge la fine dell'array fisico, "salta" alla posizione 0. Matematicamente, questo è gestito tramite l'operatore **modulo**:

$$\text{nuova_posizione} = (\text{posizione_attuale} + 1) \pmod{\text{lunghezza_array}}$$

Questa struttura permette operazioni di inserimento e rimozione in **tempo costante** $O(1)$.

Sebbene circolare, l'array ha comunque una dimensione fisica limitata. Quando l'array è pieno (ovvero quando **head** == **tail**), Java deve allocare un nuovo array più grande e copiare gli elementi. Tuttavia, questa operazione avviene raramente se la capacità iniziale è ben calibrata.

A.4 La Struttura Dati Heap

Lo **Heap** (o cumulo) è una struttura dati basata su albero che soddisfa proprietà specifiche di forma e ordine. Nonostante sia concettualmente un albero binario, la sua implementazione più efficiente e comune avviene tramite un **array dinamico**, eliminando l'overhead dei puntatori.

Un Heap deve rispettare due vincoli strutturali:

1. **Proprietà della Forma**: L'albero deve essere un **albero binario quasi completo**. Ciò significa che tutti i livelli sono completamente riempiti, ad eccezione eventualmente dell'ultimo, che deve essere riempito da sinistra a destra.
2. **Proprietà dello Heap (Ordine)**:
 - **Min-Heap**: Il valore di ogni nodo è maggiore o uguale al valore del suo nodo padre. Di conseguenza, la radice contiene il valore minimo. (Implementazione di default della `PriorityQueue` in Java).
 - **Max-Heap**: Il valore di ogni nodo è minore o uguale al valore del suo nodo padre. La radice contiene il valore massimo.

La genialità dello heap risiede nella sua mappatura implicita in un array. Dato un elemento memorizzato all'indice i (partendo da 0), è possibile calcolare la posizione dei suoi parenti tramite semplici operazioni aritmetiche:

- **Figlio sinistro**: $2i + 1$
- **Figlio destro**: $2i + 2$
- **Padre**: $\lfloor (i - 1) / 2 \rfloor$

Questa rappresentazione garantisce la **contiguità della memoria**, rendendo lo heap estremamente veloce grazie alla *cache locality*.

Operazioni Principali

Le operazioni devono mantenere intatte le due proprietà sopra descritte. Quando un nuovo elemento viene aggiunto:

1. Viene inserito nella prima posizione libera (l'ultima cella dell'array) per mantenere la forma dell'albero.
2. Viene confrontato con il padre. Se la proprietà dell'ordine è violata (es. nel Min-Heap il nuovo elemento è più piccolo del padre), i due vengono scambiati.
3. Il processo continua risalendo l'albero finché la proprietà non è ripristinata.

Complessità: $O(\log n)$, corrispondente all'altezza dell'albero.

Rimuovere la radice (l'elemento con priorità massima) è l'operazione cardine:

1. Si preleva il valore della radice.
2. L'ultimo elemento dell'array (il più a destra dell'ultimo livello) viene spostato nella radice.
3. Si confronta la nuova radice con i suoi figli. Se viola la proprietà dell'ordine, viene scambiata con il figlio minore (nel caso del Min-Heap).
4. L'elemento "affonda" lungo l'albero finché non trova la sua corretta collocazione.

Complessità: $O(\log n)$.

Analisi delle Prestazioni:

Operazione	Complessità	Descrizione
<code>peek()</code>	$O(1)$	Accesso immediato alla radice.
<code>add()</code>	$O(\log n)$	Inserimento con risalita.
<code>poll()</code>	$O(\log n)$	Estrazione con ripristino (heapify).
<code>buildHeap</code>	$O(n)$	Creazione di uno heap da un array disordinato.

Tabella A.1: Complessità computazionale dello Heap

Osservazione A.1. Sebbene la ricerca di un elemento generico sia $O(n)$ (perché lo heap non è un albero binario di ricerca), esso è imbattibile per la gestione delle priorità, dove l'unico interesse è accedere rapidamente all'elemento "più importante".

A.5 La Coda a Priorità (Priority Queue)

La **Coda a Priorità** è un Tipo di Dato Astratto (ADT) che estende il concetto di coda classica. Mentre in una coda standard (FIFO, si pensi ad esempio alla fila al supermercato) l'ordine di uscita è determinato esclusivamente dal tempo di arrivo, in una coda a priorità ogni elemento è associato a un valore di **priorità** (si pensi ad esempio alla fila al pronto soccorso).

In una coda a priorità, l'ordine di estrazione segue queste regole:

1. Un elemento con priorità maggiore viene servito prima di un elemento con priorità minore.
2. Se due elementi hanno la stessa priorità, vengono serviti secondo il loro ordine nella coda (solitamente FIFO, sebbene non sia sempre garantito da tutte le implementazioni).

Le operazioni fondamentali definite dall'ADT sono:

- `insert(item, priority)`: Aggiunge un elemento con una data priorità.
- `peek()`: Restituisce l'elemento con la priorità massima (o minima) senza rimuoverlo.

- `pull()` (o `pop`): Rimuove e restituisce l'elemento con la priorità massima (o minima).

La scelta standard per implementare una coda a priorità è un Min-heap (vedi paragrafo precedente) in cui la "priorità" è determinata dal valore stesso dell'elemento (o da un `Comparator`).